

Recipix — Language Specification (v4.1)

This is v4.1 of the Recipix specification. It supersedes v4 by applying three final decisions: `quantity_of` as a unary operator (#34), `substitute` slots restricted to bare identifiers (#35), and a reworded parameter-passing decision (#16).

With v4.1 in place, the language design is **locked**. Next phase: A2 (EBNF grammar).

Replace `docs/00_project_briefing.md` with this document.

Overview

Recipix is a domain-specific programming language for describing, parameterizing, scaling, substituting, and varying recipes. Every ingredient quantity carries a unit (`200 g`, `500 ml`, `2 count`, `180 °C`, `30 min`), and the type system enforces dimensional correctness at compile time.

The language has user-defined parameterized recipes, scalar helper functions, conditionals, count-bounded and iterator loops, lists, and three domain-specific operations (`evaluate`, `scale`, `substitute`).

The exam-defining answer

"What makes Recipix a domain-specific language and not a generic scripting language?"

Dimensional type discipline. Every numeric quantity carries a dimension (mass, volume, count, temperature, duration). The type system catches at compile time any operation that mixes dimensions — adding grams to milliliters, multiplying two masses together, substituting a volume for a mass. A general-purpose language treats `200` and `500` as raw numbers and lets the programmer combine them into nonsense. Recipix lifts the dimensional structure into the type system, making the nonsense impossible to express. The domain-specific operations (`evaluate`, `scale`, `substitute`) and the structured types (recipes, lists) are all built around preserving this invariant.

1. Tokens and lexical structure

Tokenization rules

- Whitespace separates tokens. Comments start with `//` and run to end-of-line.
- String literal contents are **UTF-8 encoded**; any character except `"` and newline is allowed inside the quotes. No escape sequences in v1.
- Identifiers match `[a-zA-Z_][a-zA-Z0-9_]*`. ASCII only. Hyphens are not allowed in identifiers.
- Quantities are **two tokens**: a numeric literal followed by a unit keyword. Within a quantity literal, the number and unit may be separated by any amount of whitespace and/or `//` comments, but **no other tokens may appear between them**. Single space is the recommended style.
 - Valid: `200 g`, `1.5 kg`, `2 count`, `180 °C`, `1 pinch`, `200 g`, `200 /* heavy */ g`
 - Invalid: `200g` (no separator — lexical error), `200 + g` (other token between — parse error)
- The `°C` token is treated as a single multi-character token. The degree symbol cannot appear standalone.

Token categories

Category	Examples
Identifier	<code>flour, pancakes, vegan</code>
Integer literal	<code>200, 0, 42</code>
Float literal	<code>1.5, 0.75, 3.14</code>
String literal	<code>"Cook", "Hamuru karıştır" (UTF-8)</code>
Reserved keyword	<code>recipe, function, ingredient, step, let, if, else, repeat, times, foreach, in, at, for, evaluate, scale, substitute, serves, with, by, ratio, return, true, false, quantity_of</code>
Type-name keyword	<code>int, float, bool, Mass, Volume, Count, Temperature, Duration, Pinch</code>
Action-verb keyword	<code>combine, mix, pour, melt, whisk, blend, bake, flip, add, sprinkle, drizzle, knead</code>
Unit keyword	<code>mg, g, kg, ml, l, tsp, tbsp, cup, °C, min, hr, count, pinch</code>
Operator	<code>+, -, *, /, ==, !=, <, <=, >, >=, &&, `</code>
Separator	<code>(,), {, }, [,], ,, :, -></code>

`string` is **not** a primitive type. String literals appear only in fixed syntactic positions (step descriptions). They are a token category, not a type that admits operations.

`quantity_of` is a reserved keyword that introduces a unary operator (see §2 and §5). It is not a function name and cannot be redefined or referenced as a value.

Note on the closed type-name set: Recipix does not have user-facing type-parameter syntax. There is no `Quantity<Mass>` or `List<T>` in user programs — only the type-name keywords above. List type annotations are not allowed in user programs in v1; list types are inferred at literal construction and at `foreach` loop entry. The angle-bracket forms (`Quantity<D>`, `List<T>`) appear only in this specification document as meta-language, never in source code.

2. Types

Primitive types

- `int` — 64-bit signed integer.
- `float` — IEEE 754 double.
- `bool` — `true` or `false`.

Domain primitives

- **Quantity types**, one per dimension. In user programs they are written `Mass`, `Volume`, `Count`, `Temperature`, `Duration`. In this spec we write `Quantity<D>` as meta-language shorthand for the family.

- **Pinch** — a separate primitive type, **not** a `Quantity<D>`. Pinch values are constructed by the literal `1 pinch` (the integer prefix is required syntactically but ignored semantically; `1 pinch` and `5 pinch` denote the same singleton ceremonial value). Pinch supports **no arithmetic, no comparison, no scaling, no substitution-by-ratio**. The type checker rejects any expression in which a `Pinch` value appears outside an ingredient declaration or a step-action argument list. Rationale: not every cooking quantity is a measurement. "A pinch of salt" is a gesture, not a number. Modeling this in the type system rather than fudging it with `mg` keeps the language honest about what it knows and does not know.

Structured types

- **Ingredient** — a structural record with two fields: `name` (a label, not an identifier) and `quantity` (a `Quantity<D>` or `Pinch`). Ingredient *identity in the program* is the binding (the identifier in the recipe's symbol table); the `name` field carries a human-readable label and equals the identifier text by default. Two `Ingredient` values have the same type if they have the same field shape (structural equivalence).
- **Recipe** — a record `{ name: string-label, servings: int, ingredients, steps }`. Recipes use **name equivalence**: two recipes have the same recipe-type only if they were declared with the same name.
- **Step** — a record `{ description: string-literal, temperature?: Temperature, duration?: Duration, actions }` (the `?` marks optional fields).
- **List<T>** — homogeneous list of `T`. All elements must have the same type. Lists of quantities must all share the same dimension. The list type is *inferred* at literal construction; users do not write list type annotations in v1.

Implicit projection: Ingredient → Quantity in arithmetic context

When an `Ingredient` value appears in an arithmetic, comparison, or substitution context that expects a `Quantity<D>`, it implicitly projects to its `quantity` field. Equivalently, the user may write `quantity_of(<ingredient>)` for an explicit projection.

Examples:

- Inside a step, `flour + 100 g` is shorthand for `quantity_of(flour) + 100 g`.
- `quantity_of(flour)` is the explicit form, recommended for clarity in non-trivial arithmetic.

This is a deliberate ergonomic decision; without it, every arithmetic expression in a recipe would need explicit projection. Trade-off accepted: a small loss of type-system uniformity in exchange for substantial readability gain.

`quantity_of` is a unary operator, not a function

The explicit projection form `quantity_of(<expr>)` is a **unary operator**, not a function call. It is recognized by the parser as a dedicated production, not as a name lookup. Users cannot redefine it, assign it to a variable, or pass it as an argument. Its operand must have type `Ingredient`; its result type is the dimension of that ingredient's quantity field (`Mass`, `Volume`, `Count`, `Temperature`, `Duration`, or `Pinch`).

Trade-off accepted: a small departure from syntactic uniformity (it looks like a call but isn't one) in exchange for a cleaner type system — `quantity_of` does not need a `FunctionType` representation, since no function

type can express "argument is **Ingredient**, return type is its quantity field's dimension." This is type-level structure that the **function** declaration form, with its fixed return-type annotation, cannot express.

Type equivalence (summary)

- **Structural** for **Quantity<D>** (any two **Mass** values are the same type, regardless of unit), for **Ingredient**, and for **List<T>**.
- **Name** for **Recipe**.
- This split is deliberate: dimensional correctness is what matters for quantities (units are convertible representations of the same underlying dimension), but recipe identity is what matters for recipes (**pancakes** and **crepes** should not be interchangeable just because they happen to share an ingredient set).

3. Arithmetic on quantities

Numeric (int/float) arithmetic

Operation	Result
<code>int + int, int - int, int * int</code>	<code>int</code>
<code>int / int</code>	<code>int</code> (truncated toward zero)
Any binary op with one <code>int</code> and one <code>float</code>	<code>float</code> (the <code>int</code> is implicitly widened)
<code>float op float</code>	<code>float</code>
Implicit <code>int</code> → <code>float</code> coercion	Allowed only in mixed-mode arithmetic
Implicit <code>float</code> → <code>int</code> coercion	Never allowed; a Part-2 builtin <code>to_int(x)</code> may be added for explicit conversion

Quantity arithmetic

Operation	Result
<code>mass + mass, volume + volume, etc.</code> (same dimension)	Allowed, with implicit unit conversion. <code>200 g + 1 kg</code> evaluates to <code>1200 g</code> .
<code>mass + volume, temperature + duration, etc.</code> (different dimensions)	Compile-time type error
<code>quantity - quantity</code> (same dimension)	Allowed
<code>quantity * scalar</code>	Dimension preserved. <code>200 g * 2</code> evaluates to <code>400 g</code> .
<code>scalar * quantity</code>	Same as <code>quantity * scalar</code> . Multiplication is commutative on scalar/quantity pairs. <code>2 * 200 g</code> evaluates to <code>400 g</code> .

Operation	Result
<code>quantity * quantity</code> (any dimensions)	Type error — no dimensional products in v1
<code>quantity / scalar</code>	Dimension preserved
<code>quantity / quantity</code> (same dimension)	Plain unitless number. <code>1 kg / 200 g</code> evaluates to <code>5</code> .
<code>quantity / quantity</code> (different dimensions)	Type error
Any operation involving a <code>Pinch</code> value	Type error (ceremonial discipline)
Unary <code>-</code> on a quantity	Allowed syntactically; produces <code>(-1) * quantity</code> . The interpreter accepts negative quantities; the type checker does not flag them in v1.

Constructing a quantity from a computed value

Because quantity literals are two tokens (a numeric literal followed by a unit keyword), an *expression* cannot be followed by a unit keyword to produce a quantity. To construct a quantity from a computed numeric value, multiply by a one-unit literal of the desired unit:

```
ingredient eggs : half(servings) * 1 count
ingredient flour : 50 g * servings
```

This is a deliberate consequence of decision #7 (two-token quantity literals). Trade-off accepted: slightly more verbose construction in exchange for unambiguous lexing.

Conversion factors

Dimension	Base unit	Conversions
Mass	<code>g</code>	<code>1 kg = 1000 g</code> , <code>1 mg = 0.001 g</code>
Volume	<code>ml</code>	<code>1 l = 1000 ml</code> , <code>1 tsp = 5 ml</code> , <code>1 tbsp = 15 ml</code> , <code>1 cup = 240 ml</code>
Count	<code>count</code>	(no conversions)
Temperature	<code>°C</code>	(no conversions; <code>°C</code> is the only unit)
Duration	<code>min</code>	<code>1 hr = 60 min</code>

Quantities are normalized to base units at runtime for uniform arithmetic.

4. Boolean expressions

- Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`.

- Allowed between two values of the same type.
- Comparing values of different types is a type error.
- For `Quantity<D>`, comparison is allowed only between two quantities of the same dimension; values are compared after normalization to the base unit.
- Comparisons involving `Pinch` are not allowed.
- Logical: `&&`, `||`, `!`. Short-circuit evaluation for `&&` and `||`, left-to-right.

5. Operator precedence

From highest to lowest:

Level	Operators	Associativity
1	unary <code>-</code> , <code>!</code> , <code>quantity_of(...)</code>	right
2	<code>*</code> , <code>/</code>	left
3	<code>+</code> , <code>-</code> (binary)	left
4	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	non-associative
5	<code>==</code> , <code>!=</code>	non-associative
6	<code>&&</code>	left
7	<code>`</code>	

Parenthesization overrides precedence. Operand evaluation order is **left-to-right and fully defined** (Sebesta §7.6).

Note: `quantity_of(...)` is a unary operator at level 1, not a function call. See §2 for its type rule.

6. Names, binding, scope, lifetime

Identifier rules

Identifiers match `[a-zA-Z_][a-zA-Z0-9_]*`, are case-sensitive, ASCII only, and must not collide with reserved keywords, type-name keywords, action-verb keywords, or unit keywords.

Ingredient identity

An ingredient declaration `ingredient flour : 200 g` introduces a binding named `flour` in the enclosing recipe's symbol table. **Identity is the binding** — when later code references `flour`, it resolves through the symbol table. The `Ingredient` record's `name` field is a human-readable label (defaulting to the identifier text), not the program-level identity. Likewise, `substitute(r, flour, ...)` looks up the binding `flour` in the recipe's ingredient symbol table; it does not match against the `name` field's string contents.

Binding times

Binding	Bound at compile time	Bound at run time
Recipe declarations (name and signature)	✓	
Function declarations (name and signature)	✓	
Ingredient <i>types</i>	✓	
Ingredient <i>quantity values</i>		✓ (evaluated at recipe instantiation)
Recipe parameters		✓ (at instantiation)
Function parameters		✓ (at call)
Loop variables (foreach)		✓ (at iteration entry)
let -bound names — type	✓ (annotation required)	
let -bound names — value		✓ (initializer evaluated when control reaches declaration)

Scoping

Static (lexical) scoping throughout. Each of the following constructs opens a fresh scope:

- A **recipe** body (with its parameters and ingredients in scope)
- A **function** body (with its parameters in scope)
- A **step** body (with the recipe's ingredients in scope)
- An **if** block and its **else** block (each independently)
- A **repeat** body
- A **foreach** body (with the loop variable in scope)

Inner scopes can read outer-scope names. **No shadowing**: declaring an identifier already visible in any enclosing scope is a compile-time error.

Single-assignment discipline

- Ingredient names within a recipe scope: declared exactly once.
- **let**-bound names within their enclosing scope: declared exactly once. Immutable after binding.
- Recipe parameters and function parameters: bound once at call/instantiation time, immutable thereafter.
- Loop variables (**foreach x in ...**): rebound on each iteration; within a single iteration, immutable.

Lifetime

Entity	Lifetime
Recipe declarations	Static (whole program)

Entity	Lifetime
Function declarations	Static (whole program)
Recipe parameters	Stack-dynamic, per instantiation
Function parameters	Stack-dynamic, per call
Ingredients within a recipe	Stack-dynamic, per recipe instantiation (parsed at decl, type-checked at decl, evaluated at instantiation)
<code>let</code> -bound names	Stack-dynamic, per enclosing scope; immutable after initializer evaluates
Loop variables	Stack-dynamic, per iteration

`scale` and `substitute` are functional

These operations produce new recipe values; they do not mutate the original. `scale(pancakes, by: 2)` returns a fresh `Recipe` value with all relevant quantities doubled; `pancakes` itself is unchanged. Substitution and scaling are **call-site only** — they cannot appear inside a recipe body. There is no `this` keyword; a recipe cannot reference itself during construction. Every Recipix expression is referentially transparent in v1.

Parameter passing

All parameter passing in Recipix is **semantically by-value**: the callee receives values that the caller cannot observe being modified, because no Recipix operation modifies any value. The implementation copies primitives and quantities; for recipes and lists it shares immutable references to avoid unnecessary copying. Because Recipix has no mutation, this implementation choice is unobservable — Sebesta §9.5 distinguishes by-value, by-reference, by-result, and by-value-result modes by their *observable* effects, and in v1 all of these collapse to a single mode. If Recipix v2 introduces mutation (e.g., a mutable accumulator inside a `repeat` body), this decision will be the first thing the language design has to revisit.

7. Declarations

Recipe declaration (parameterized)

```
recipe <name>([<params>]) serves <expr> {
  <ingredient_decls>
  <step_decls>
}
```

- `<params>` is a comma-separated list of `<ident> : <type>` pairs (zero or more).
- `<expr>` after `serves` is an `int`-typed expression. The expression is parsed greedily up to the opening `{` of the body. Usually a literal, but may use parameters: `recipe pancakes(people: int) serves people`.
- A recipe with zero parameters is declared as `recipe pancakes() serves 4 { ... }` and called as `pancakes()`.

Function declaration

```
function <name>([<params>]) -> <return_type> {
  <statements>
  return <expr>
}
```

- For scalar helpers operating on primitives and quantities. Functions cannot return **Recipe** values in v1.
- **<return_type>** is one of the primitive types or one of the quantity type names (**Mass**, **Volume**, etc.).
- Single **return** statement at the end of the body. No early return in v1.

Ingredient declaration

```
ingredient <name> : <quantity_or_expr>
```

- Ingredient names are scoped to the enclosing recipe.
- The right-hand side is any expression of type **Quantity<D>** or **Pinch**. Usually a literal (**200 g**); may reference recipe parameters or call functions (**flour : 50 g * servings**, **eggs : half(servings) * 1 count**).

let declaration

```
let <name> : <type> = <expr>
```

- Binds an immutable name in the enclosing scope.
- May appear inside a **recipe** body, **function** body, **step** body, or any block (**if**, **repeat**, **foreach**).
- May **not** appear at the top level (outside any recipe or function).
- Type annotation is **required** in v1. No type inference.

Step declaration

```
step <string_lit> [ at <expr> ] [ for <expr> ] {
  <step_actions>
}
```

- **at** modifier: optional, takes a **Temperature** expression.
- **for** modifier: optional, takes a **Duration** expression.
- Modifier order is fixed: **at** precedes **for**. Each is independently optional. All four combinations are legal.
- Step actions are themselves statements (see §8).

8. Statements

Step actions (uniform function-call form)

All step actions are function calls. There are no parens-free statement forms.

The closed set of action verbs in v1: `combine`, `mix`, `pour`, `melt`, `whisk`, `blend`, `bake`, `flip`, `add`, `sprinkle`, `drizzle`, `knead`. Action verbs are reserved keywords. The set is closed in v1 — users cannot define new step actions. (User-defined functions cover scalar logic; step actions cover physical operations and need to be visible to the type checker.)

Action verbs are an exception to the homogeneous-list rule

A statement like `combine(flour, salt)` mixes ingredients of different dimensions (Mass and Pinch) into one argument list. Under the homogeneous-list rule (§2), this would be a type error. **Action verbs are explicit exceptions to that rule.** The action verb signature table (locked in Part 2 H6) defines, per-verb, what argument types are accepted; for combining and mixing verbs, heterogeneous ingredient lists are accepted.

This is a deliberate design carve-out: action verbs are language built-ins with their own signature rules, and ordinary list rules do not apply to their argument lists. Trade-off accepted: small loss of type-system uniformity, in exchange for natural expression of cooking actions.

`let` statement

```
let <name> : <type> = <expr>
```

(Same form as in declarations; usable as a statement inside any block.)

Conditional statement

```
if <bool_expr> {
  <statements>
}
[else {
  <statements>
}]
```

Braces around both `if` and `else` bodies are **mandatory**. This eliminates the dangling-`else` ambiguity by construction.

Count-bounded loop

```
repeat <int_expr> times {
  <statements>
}
```

- The expression must evaluate to a non-negative `int`.
- A negative value is a runtime error.

Iterator loop

```
foreach <name> in <list_expr> {  
  <statements>  
}
```

- `<list_expr>` must be of type `List<T>` for some `T`.
- The loop variable is bound to type `T` in the body; the type is inferred from the list's element type.
- Empty lists produce zero iterations (no error).

Function `return` statement

```
return <expr>
```

- Appears only as the last statement of a function body.

9. Domain-specific operations

All three operations are **call-site only**. They cannot appear inside a recipe body.

`evaluate`

```
evaluate <recipe_expr>
```

Type-checks and elaborates the recipe expression into a final printable recipe value with all quantities normalized.

`scale`

```
scale(<recipe_expr>, by: <scalar_expr>)
```

- Returns a fresh `Recipe` value.
- Multiplies every Mass, Volume, and Count quantity in the recipe by the scalar.
- **Does not** scale Temperature or Duration quantities.
- Multiplies the `servings` field by the scalar (rounded to int).
- Sub-recipes are not scaled in v1 (no recipe composition).

`substitute`

```
substitute(<recipe_expr>, <ingredient_name>, with: <ingredient_name>, ratio:
```

```
<scalar_expr>)
```

- Returns a fresh **Recipe** value.
- The two **<ingredient_name>** slots are **bare identifiers**, not general expressions. They are resolved at compile time as lookups in the recipe's ingredient binding table. Writing an arbitrary expression in either slot is a parse error.
- The **<scalar_expr>** slot accepts any **int** or **float** expression.
- Looks up the original **<ingredient_name>** in the recipe's ingredient binding table; replaces its binding with the replacement and applies the ratio: new quantity = **original_quantity** * **ratio**.
- Both ingredients must have quantities of the **same dimension** — substituting a Mass for a Volume is a compile-time type error. Substituting any value for a **Pinch** (or vice versa) is also a type error, since **Pinch** is not a **Quantity<D>**.
- The replacement ingredient name must be defined in scope at the substitute call site (typically as another ingredient inside the recipe being substituted).

The bare-identifier restriction makes Decision #28 (ingredient identity is the binding) visible in the grammar itself, rather than enforcing it later in the type checker. Trade-off accepted: small loss of expressive uniformity (you cannot programmatically compute which ingredient to substitute) in exchange for grammar-level clarity about what **substitute** actually does — symbol-table lookup, not runtime value matching.

10. Errors caught

Compile-time (type checker / parser)

1. Dimension mismatch in arithmetic, comparison, or substitution
2. Pinch value in arithmetic, comparison, or scaling/substitution-by-ratio position (ceremonial discipline)
3. Heterogeneous list literal (e.g., **[200 g, 500 ml]**)
4. Unknown identifier (use before declaration)
5. Single-assignment violation (re-declaring a name in the same scope)
6. Shadowing (declaring a name visible in an outer scope)
7. Type mismatch in **if** condition (must be **bool**)
8. Type mismatch in **repeat** count (must be **int**)
9. Type mismatch in **foreach** source (must be a list)
10. Wrong type for **at** modifier (must be **Temperature**)
11. Wrong type for **for** modifier (must be **Duration**)
12. Wrong arity in recipe or function call
13. Wrong argument types in recipe or function call
14. Unknown ingredient referenced in a step or in **substitute**
15. Implicit **float** → **int** coercion attempt
16. Substitution between **Pinch** and a **Quantity<D>** (different primitive types)
17. Non-identifier expression in either ingredient-name slot of **substitute** (parse error)
18. **quantity_of** applied to a non-**Ingredient** operand

Run-time (interpreter)

1. Negative **repeat** count

2. Negative scaling factor
3. Zero scaling factor
4. Substitution of a non-existent ingredient name (when computed at runtime — rare, since slots are bare identifiers checked at compile time)
5. Division by zero in scalar arithmetic

Each error reports line number and a useful message.

11. Sample programs

Sample 1 — A recipe using a helper function

```
function half(n: int) -> int {
  return n / 2
}

recipe pancakes(servings: int) serves servings {
  ingredient flour : 50 g * servings
  ingredient milk  : 60 ml * servings
  ingredient eggs  : half(servings) * 1 count
  ingredient salt  : 1 pinch

  step "Mix dry" {
    combine(flour, salt)
  }
  step "Add wet" {
    combine(milk, eggs)
  }
  step "Cook batches" at 180 °C for 3 min {
    repeat servings times {
      pour(milk)
      flip()
    }
  }
}

evaluate scale(pancakes(servings: 4), by: 1.5)
```

Notes on this sample:

- `half(servings) * 1 count` constructs a `Count` quantity from a computed integer (§3, "Constructing a quantity from a computed value").
- `combine(flour, salt)` mixes a `Mass`-typed ingredient and a `Pinch`-typed ingredient. This is allowed because action verbs are exceptions to the homogeneous-list rule (§8).
- `int / 2` in `half` is integer division, truncating toward zero (§3).

Sample 2 — Substitution at the call site (no `this`)

```

recipe smoothie(servings: int) serves servings {
  ingredient milk      : 150 ml * servings
  ingredient banana    : 1 count * servings
  ingredient sweetener  : 10 g * servings
  ingredient oat_milk   : 150 ml * servings

  step "Blend everything" for 1 min {
    combine(milk, banana, sweetener)
    blend()
  }
}

// Vegan variant: substitute happens at the call site, not inside the recipe.
evaluate substitute(smoothie(servings: 2), milk, with: oat_milk, ratio: 1.0)

// Non-vegan variant:
evaluate smoothie(servings: 2)

```

Notes on this sample:

- `milk` and `oat_milk` in the `substitute` call are bare identifiers — symbol-table lookups in the smoothie recipe's ingredient bindings (§9 substitute, Decision #35).
- The `oat_milk` ingredient is declared inside the recipe so that it exists as a binding the `substitute` call can reference. This is a deliberate consequence of the no-`this` decision: alternative ingredients are pre-declared in the recipe, and the choice between them is made at the call site.

Sample 3 — A type-error program (for D3 P2)

```

recipe broken() serves 1 {
  ingredient flour : 200 g
  ingredient water : 100 ml

  step "Combine wet and dry" {
    // ERROR (line below): cannot add Mass and Volume.
    // The implicit ingredient-to-quantity projection turns
    // `flour` into a Mass and `water` into a Volume, then
    // arithmetic fails the dimension-match check.
    let total : Mass = flour + water
  }
}

evaluate broken()

```

The compile-time error is at the `let` line: implicit projection turns `flour + water` into `Mass + Volume`, which is a dimension mismatch. The type checker catches this before execution.

12. The 35-decision design table (v4.1)

Rows changed in v4.1 are marked **[v4.1 REVISED]** or **[v4.1 NEW]**. Rows added in v4 (over v3) are marked **[v4 NEW]** or **[v4 REVISED]**.

#	Decision	Choice	Sebesta
1	Primitive types	<code>int</code> , <code>float</code> , <code>bool</code>	§6.2
2	String handling	Token category, not a type	§6.2
3	Domain primitives	Quantity types <code>Mass</code> , <code>Volume</code> , <code>Count</code> , <code>Temperature</code> , <code>Duration</code>	§6.2
4	Ceremonial primitive [v4 REVISED]	<code>Pinch</code> is its own primitive type, separate from quantities; no arithmetic, no comparison, no scaling	§6.2 (deliberate trade-off)
5	Structured types	<code>Ingredient</code> , <code>Recipe</code> , <code>Step</code> , <code>List<T></code>	§6.5–6.7
6	List discipline	Homogeneous; quantity lists share one dimension	§6.5
7	Quantity literals	Two tokens, separator (whitespace and/or comments) required between number and unit	§4.2
8	Strong typing	Yes, strict	§6.12
9	Implicit coercion [v4 REVISED]	Within-dimension unit coercion (<code>g</code> ↔ <code>kg</code> , etc.); plus <code>int</code> widens to <code>float</code> in mixed-mode arithmetic; never <code>float</code> → <code>int</code> ; never across dimensions	§7.4
10	Type equivalence	Structural for <code>Quantity<D></code> , <code>Ingredient</code> , <code>List<T></code> ; name for <code>Recipe</code>	§6.14
11	Scoping	Static (lexical); each block opens a fresh scope	§5.5
12	Shadowing	Forbidden; redeclaration of a visible name is an error	§5.5
13	Single-assignment	Ingredients, <code>let</code> , parameters: immutable	§5.4
14	Lifetime	Recipes/functions static; everything else stack-dynamic	§5.4
15	User-defined abstractions	Parameterized recipes (primary) + scalar <code>function</code> (helpers)	§9
16	Parameter passing [v4.1 REVISED]	Semantically by-value for all types. Recipix has no mutation, so by-value and by-reference are indistinguishable at the language level. The implementation passes primitives and quantities by copy, and recipes and lists by shared immutable reference, purely as an optimization — programs cannot detect the difference because no operation mutates the referenced value. If a future Recipix version introduces mutation, this decision will need to be revisited.	§9.5
17	Operator precedence and	Standard hierarchy; comparison non-associative	§7.2

#	Decision	Choice	Sebesta
	associativity		
18	Operand evaluation order	Left-to-right, fully defined	\$7.6
19	Short-circuit	Yes for <code>&&</code> and <code>`</code>	
20	Conditional	<code>if <bool> { ... } [else { ... }]</code> ; braces mandatory; dangling-else eliminated by construction	\$3.4, \$8.1
21	Loops	<code>repeat <int> times { ... }, foreach <id> in <list> { ... }</code>	\$8.2, \$8.3
22	Step action syntax	Uniform function-call form: <code>combine(a, b, c), pour(x), flip()</code>	(consistency)
23	Step modifiers	<code>[at <expr>] [for <expr>]</code> ; both optional, fixed order	(grammar simplicity)
24	Unary minus	Exists; binds tighter than <code>*</code> and <code>/</code> ; allowed on <code>int</code> , <code>float</code> , <code>Quantity<D></code>	\$7.2
25	<code>scale / substitute</code> semantics	Functional: produce new values, do not mutate. Call-site only; no <code>this</code> keyword.	\$9 (referential transparency)
26	Composition (recipe-in-recipe)	Removed from v1	(scope)
27	Assignment	Statement, not expression. <code>let</code> is the only binding form.	\$7.6
28	Ingredient identity [v4 NEW]	The binding (identifier in symbol table). The <code>name</code> record field is a label, not the program-level identity. <code>substitute(r, flour, ...)</code> resolves through the symbol table.	\$5.2, \$6.7
29	Action verbs and homogeneous lists [v4 NEW]	Action verbs are explicit exceptions to the homogeneous-list rule. Per-verb signatures (locked in Part 2 H6) define accepted argument types; combining/mixing verbs accept heterogeneous ingredient lists.	\$6.5 (deliberate carve-out)
30	Constructing quantities from expressions [v4 NEW]	An expression cannot be followed by a unit keyword. Use <code><expr> * 1 <unit></code> to construct a quantity from a computed value. Consequence of decision #7.	\$4.2, \$6.2
31	Ingredient → Quantity projection [v4 NEW]	When an <code>Ingredient</code> value appears in arithmetic, comparison, or substitution context, it implicitly projects to its <code>quantity</code> field. Equivalent explicit form: <code>quantity_of(<ingredient>)</code> .	\$6.7 (deliberate ergonomic trade-off)
32	Type-name keywords (no	User programs use <code>Mass</code> , <code>Volume</code> , <code>Count</code> , <code>Temperature</code> , <code>Duration</code> , <code>Pinch</code> , <code>int</code> , <code>float</code> , <code>bool</code> as type names. No angle-	\$6.5

#	Decision	Choice	Sebesta
	type-parameter syntax) [v4 NEW]	bracket type parameters in user syntax. List type annotations are not allowed in v1; list types are inferred.	
33	String literal encoding [v4 NEW]	UTF-8. Any character except " and newline allowed. No escape sequences in v1.	\$4.2
34	quantity_of operator status [v4.1 NEW]	Unary operator (level-1 precedence, right-associative), not a function. Cannot be redefined, assigned to a variable, or passed as an argument. Type rule: Ingredient → <dimension of quantity field> . This avoids needing a FunctionType whose return type depends on the argument's type-level structure.	\$6.7, \$7.2
35	substitute argument forms [v4.1 NEW]	The two ingredient-name slots are bare identifiers (symbol-table lookups), not general expressions. Restricting them syntactically makes the binding-resolution rule (Decision #28) visible in the grammar. The ratio slot remains a general scalar expression.	\$5.2, \$6.7

Signature decisions to memorize for the exam: #4 (ceremonial Pinch as its own primitive), #8 (strong typing), #9 (split coercion: within-dimension unit AND int-widens-to-float), #10 (split equivalence rule), #15 (parameterized recipes as primary abstraction), #16 (by-value semantically; references as implementation detail in absence of mutation), #20 (mandatory braces), #25 (functional scale/substitute, no **this**), #29 (action verbs as homogeneous-list exception), #31 (Ingredient → Quantity implicit projection), #34 (**quantity_of** as operator), #35 (substitute slots as identifiers).

13. Grammar productions ready for A2

The following productions are pre-committed to be in A2 (EBNF). They encode v4.1 decisions that have grammar-level consequences:

Unary expression (decision #34)

```
<unary>      ::= "-" <unary>
               |  "!" <unary>
               |  "quantity_of" "(" <expr> ")"
               |  <primary>
```

quantity_of is at the unary level (level 1), parsed by a dedicated production. It is not reachable through **<primary>** (which handles function calls, identifiers, literals, etc.).

Substitute call (decision #35)

```

<substitute_call> ::= "substitute" "(" <expr> ","
                    IDENT ","
                    "with" ":" IDENT ","
                    "ratio" ":" <expr>
                    ")"

```

The two ingredient-name slots are **IDENT** terminals — bare identifiers — not **<expr>**. The grammar makes the restriction visible.

Quantity-literal lookahead (decision #7, recap)

```

<primary> ::= INT_LIT [ <unit_keyword> ]
          |  FLOAT_LIT [ <unit_keyword> ]
          |  ...

```

A numeric literal optionally followed by a unit keyword forms a quantity literal. The optional unit keyword must be lexically the next token after the numeric literal (only whitespace and comments may intervene).

14. AST nodes ready for C1

The following AST nodes are pre-committed for the implementation phase:

```

@dataclass
class QuantityOf:          # decision #34
    operand: Expr
    line: int

@dataclass
class SubstituteCall:      # decision #35
    recipe: Expr
    original_name: str      # bare identifier text, not Expr
    replacement_name: str  # bare identifier text, not Expr
    ratio: Expr
    line: int

```

Note that `SubstituteCall.original_name` and `replacement_name` are typed as `str`, not `Expr`. This is the AST-level reflection of decision #35: the grammar guarantees the slot is an identifier, so the AST does not pretend it could be anything else.

15. Summary of v4.1 changes from v4

For traceability:

Issue addressed in v4.1	Resolution
<code>quantity_of</code> as function vs. operator	Locked as a unary operator (level 1, right-associative). New decision #34. Reserved keyword. Not redefinable. New §2 paragraph; precedence-table footnote in §5; parser production and AST node committed.
<code>substitute</code> slots ambiguity	Locked as bare identifiers in the grammar, not general expressions. New decision #35. Substitute production rewritten in §9. New error #17 in §10. Parser production and AST node committed.
Decision #16 (parameter passing) wording	Reworded to honestly describe semantic-vs-implementation distinction. Now explicitly: "semantically by-value; implementation may share immutable references; the distinction is unobservable in v1 because no mutation exists." Also added as a paragraph at the end of §6.

All v3 and v4 outstanding design issues are now resolved. **The language design is locked.** Next phase: A2 (EBNF), then C1 (AST), C2 (lexer), C3 (parser).